PRN:

Batch:

Experiment no 3

Title: Parallel Reduction and Scan Algorithms using OpenMP: Dot Product and Prefix Sum

1. **Objective:**

To understand the concepts of reduction and scan (prefix sum), implement them using OpenMP, analyze their performance, and explore their applications in High-Performance Computing (HPC), Artificial Intelligence (AI), Machine Learning (ML), Scientific Computing, and Data Analytics.

2. **Use:**

Reduction and Prefix Sum are two of the most widely used parallel algorithmic patterns in High-Performance Computing. Instead of solving a single problem, these algorithms act as **building blocks** for numerous real-world applications. Modern parallel programming libraries such as **OpenMP, MPI, CUDA, cuBLAS, Intel MKL, TensorFlow, PyTorch, and Thrust** provide highly optimized implementations because these operations are executed millions or even billions of times in scientific and AI applications.

Understanding these patterns helps programmers design efficient parallel algorithms and optimize application performance.

- **Dot Product (Reduction)**

Reduction combines multiple values into a single result.

Applications

Domain	Example Application
Artificial Intelligence	Neural Network Training
Deep Learning	Fully Connected Layers
Machine Learning	Linear & Logistic Regression
Natural Language Processing	Word Embeddings, Attention Mechanism
Computer Vision	Image Similarity
Robotics	Sensor Fusion
Scientific Computing	Matrix Operations
Signal Processing	Correlation

- **Prefix Sum (Scan)**

Prefix Sum computes cumulative sums and is considered one of the most important primitives in parallel computing.

Applications

Domain	Example Application
AI	Sequence Processing
GPU Computing	Stream Compaction
Computer Graphics	Histogram Equalization
Databases	Parallel Query Processing
Graph Analytics	Breadth First Search
Image Processing	Integral Images
Parallel Sorting	Radix Sort
Big Data	Parallel Aggregation

3. **Answer the following questions before starting the programming tasks.**
    1. What is the difference between reduction and scan?
    2. Why is Dot Product easier to parallelize than Prefix Sum?
    3. Explain why synchronization is required during reduction.

Section A

Task 1: OpenMP Reduction

Implement the sum of an array using:

- Sequential implementation
- OpenMP using critical
- OpenMP using reduction

Compare

- Correctness
- Execution Time
- Speedup

Input Sizes

- 10,000
- 1,000,000
- 10,000,000

Observe the execution times.

- Why is the reduction clause faster than critical?
- Why is critical considered a bottleneck?

Task 2: Dot Product using OpenMP

Implement Dot Product using:

- Sequential approach
- OpenMP using reduction

Measure

- Execution Time
- Speedup
- Efficiency

Verify that both implementations produce the same result.

Task 3: HPC in Practice

Research how **PyTorch** or **TensorFlow** internally use matrix multiplication and dot products while training neural networks.

Write a brief note (100–150 words) explaining the role of dot products in AI.

Section B

Task 1: Prefix Sum (Scan)

Implement Prefix Sum.

- Sequential implementation
- OpenMP implementation

Compare

- Execution Time
- Speedup
- Output correctness

Observe:

Why is Prefix Sum more difficult to parallelize than Dot Product?

Identify:

- Data dependency
- Synchronization requirements

Task 2: Performance Analysis

Execute both algorithms using:

- 1 Thread
- 2 Threads
- 4 Threads
- 8 Threads
- Maximum available threads

Prepare the following table.

Threads	Dot Product Time	Prefix Sum Time	Speedup	Efficiency
1				

Threads	Dot Product Time	Prefix Sum Time	Speedup	Efficiency
2				
4				
8				

Plot:

- Threads vs Execution Time
- Threads vs Speedup

Section C:

Complete **any ONE** challenge.

Challenge 1

Implement Dot Product using

- critical
- atomic
- reduction

Compare

- Execution Time
- Scalability
- Synchronization overhead

Recommend the best implementation.

Challenge 2

Design a tree-based reduction algorithm without using the OpenMP reduction clause.

Compare it with the built-in implementation.

Challenge 3

Optimize Prefix Sum by reducing synchronization overhead.

Compare the optimized implementation with the basic implementation.

Section D: AI / HPC Case Study

Choose **any ONE** application.

- ChatGPT / Large Language Models
- Image Classification
- Recommendation Systems
- Scientific Simulation
- Weather Forecasting
- Financial Analytics

Prepare a **one-page report** covering:

1. Where is Reduction used?
2. Where is Prefix Sum used?
3. Why is parallel programming necessary?
4. Would you choose OpenMP, MPI, or CUDA? Justify your choice.

Section E: Performance Report

Complete the following table.

Algorithm	**Sequential Time**	**Parallel Time**	**Speedup**	**Efficiency**
Array Sum				
Dot Product				
Prefix Sum				

Answer the following:

1. Which implementation achieved the highest speedup? Why?
2. Which synchronization method performed best?
3. Why is Prefix Sum considered a fundamental primitive in GPU programming?
4. How would increasing the number of cores affect performance?
5. Identify one application from your specialization (AI, Cybersecurity, Blockchain, IoT, Robotics, Networking, etc.) where these algorithms can be applied.

Submit in a pdf:

- Source code for all implementations
- Execution outputs
- Performance analysis tables
- Performance graphs
- AI/HPC case study report
- Answers to the questions (total 12 questions)

